Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра програмного забезпечення

Елементи мови UML.

Методичні вказівки до виконання лабораторної роботи №6 з курсу «Вступ до інженерії програмного забезпечення»

Львів-2018

Левус Є.В.
Елементи мови UML. Методичні вказівки до виконання лабораторної роботи №6 з курсу «Вступ до інженерії програмного забезпечення»/ Левус Є.В., Нитребич О.О., Марусенкова Т.А.

Тема. Опис предметної області з використанням UML .

Мета. Навчитися створювати об’єктну модель програмної системи.

Теоретичні відомості

1. Моделювання програмних систем. Людина у будь-якій діяльності постійно користується моделями. Діти граються іграшками — зменшеними копіями реальних об’єктів. Для гри використовують не тільки готові моделі, а й створені власними руками з пластиліну, деталей конструктора. У школі діти ознайомлюються з іншими моделями: аплікація, рисунок, креслення, глобус, моделі фізичних пристроїв тощо. У подальшому житті люди також використовують моделі — макет (проект) будинку, автомобіля, моделювання фізичних, хімічних явищ та процесів.

Наявність моделі спрощує та прискорює спілкування зацікавлених осіб. Збираючись обговорити можливі рішення, люди часто занотовують основні досягнуті результати, зокрема і малюють, це природно. Простіше і швидше обговорювати проблему, маючи від чого відштовхнутися. Наприклад, пояснити дизайнеру інтер’єра чи одягу свій задум на основі його готового портфоліо («я хочу оце, тільки з такими-то змінами») краще, ніж починати опис «з нуля». Простіше сприйняти лекцію, якщо вона супроводжується слайдами, – можна задавати питання, звертаючись до готових слайдів.

Спробуємо дати означення поняттю «модель».

Модель - це спрощений вигляд реальності, креслення системи, куди може входити як детальний план, так і більш загальний вигляд. Часто говорять замість загальний - абстрактний, так би мовити, «з висоти пташиного польоту». Зазвичай, модель завжди включає елементи, які суттєво впливають на результат і не включає ті, які малозначні на даному рівні абстракції.

Модель програмного забезпечення (ПЗ) створюєтсья для того, щоб краще розуміти майбутню систему. Моделювання дозволяє вирішити різні завдання, такі як:

- візуалізація системи для замовника;
- опис структури або поведінки системи;
- одержання протипу, що дозволяє сконструювати майбутню систему;

- документація ПЗ з використанням отриманої моделі.

Чим більша і складніша програма система, тим більшого значення набуває її моделювання на етапі проектування архітектури ПЗ. Без моделі складну систему неможливо сприйняти як одне ціле.

Отже, модель програмної системи – це абстрактне представлення системи, в якому ігноруються деякі її деталі, неважливі для даного рівня розгляду системи.

Для того, щоб створити зрозумілу модель ПЗ, яку зможуть всі «прочитати» (як технічні, так і звичайні користувачі, замовники і т.д.) часто створюють словник предметної області.

Словник даних предметної області необхідний для моделювання програмної системи і використовуються для управління інформацією. У загальному випадку словник – це впорядкований набір сутностей в алфавітному порядку. Кожна сутність описується категорією та коротким поясненням. Категорія – це назва групи, сформованої за певною ознакою предметної області. Наприклад, особа, місце, механізм, об’єкт, організація і т.п.

Для складання словника розроблюваної системи використовуються класи. У словнику можуть бути абстракції, які є частиною предметної області, або класи, на які опирається реалізація. З їх допомогою описують програмні, апаратні або чисто концептуальні сутності.

При укладанні словника важливо зафіксувати виявлені відношення між сутностями типу «ціле-частина», «джерело змін – змінююча сутність», «загальна сутність – спеціалізована сутність».

Приклад короткого словника предметної області «Літаючі засоби»:

1. **Aircraft –** <u>літаючий засіб</u> (*базовий клас*); має властивості (*містить атрибути*) weight (вага), speed (швидкість), length (довжина літака), wingspan (розмах крил), model (модель літака), які притаманні всім літакам. Також може показувати інформацію про літак, його характеристики (*містить метод showInfo()*). Зв’язаний з сутністю (*класом*) Pilot, та з всіма спеціалізованими сутностями (*дочірними класами*).

2. **Military –** військові літаючий засіб. (*Отже, є похідним класом від класу Aircrafts*). Має свої спеціалізовані властивості combatMission (бойова ціль), weaponType (вид озброєння), impressionArea (область ураження).
3. **Bomber** – вид військових літаючих засобів - бомбардувальник. Є спеціалізованою сутністю Military (*похідним класом від класу Military*). Має свої спеціалізовані властивості (*свої власні атрибути*) bombAmount (кількість бомб), bombType (тип бомб).
4. **Destroyer –** вид військових літаючих засобів – винищувач. Є спеціалізованою сутністю Military (*похідним класом від класу Military*). Має свої спеціалізовані властивості (*свої власні атрибути*) bulletsAmount (кількість патронів), machineGunType (тип кулемета).
5. **Engine –** сутність двигун, що є складовою сутності Aircrafts (*відношення частина-ціле*). Має властивості (*атрибути*) engineType (тип двигуна), power (потужність двигуна).
6. **EjectorSeat** – сутність катапультуюче сидіння, що є складовою сутності Military (*відношення частина-ціле*). Має властивість (*атрибути*) available, яка вказує на наявність сидіння в літаку.
7. **Pilot –** сутність пілот. Має властивості (*атрибути*) surname (прізвище пілота), name (імя пілота), rank (звання), workExperience (досвід роботи), ability (уміння), а також поведінку, яка вказує на керування літаком (*метод drive()*).

Примітка. Курсивом в дужках наведено міркування, що стосуються програмної реалізації.

Для того, щоб змоделювати певну програмну систему, описати її функціонал, сутності та взаємозв’язки між ними часто використовують мову UML.

UML – уніфікована мова, і цей факт відображений в її назві (**Unified** Modeling Language). Часом першу літеру абревіатури помилково розшифровують як Universal, хоча до універсальності далеко, адже мова не «заточена» під

моделювання чого завгодно. В оригіналі мова розраховувалася на розробників об'єктно-орієнтованих систем. Утім, вона ж стане в нагоді і тим, хто створює якісь інші (але інформаційні) системи. «Уніфікована» означає незалежність від мови програмування. Завдяки цьому мова стає в нагоді широкому колу фахівців. Більше того, UML – це саме мова, а не, скажімо, методологія, а значить, її доволі просто «впровадити» в компанію, не порушивши заведеного порядку речей.

UML – однозначна. На відміну від природної людської мови, UML – формалізована настільки, що допускає лише єдине трактування (як математична чи хімічна формула). Нехай програміст А хоче донести певне повідомлення до програміста В. Якщо він робитиме це через UML, то по-перше, адресат прочитає в повідомленні рівно те, що у ньому було закладено, внаслідок строгої семантики мови. По-друге, програміст А бачитиме – все, що не було сказано явно, не було сказано взагалі. Оскільки є документ з інформацією, поданою стисло і недвозначно, уся команда принаймні матиме можливість зрозуміти його однаково. Як результат, зменшується ризик потоку звинувачень «Мені цього не сказали/не переказали», «Я почув щось інакше», «Я мав на увазі зовсім інше» тощо.

У UML версії 2.0 визначено 13 різних видів діаграм. Набір наочних моделей, тобто, набір діаграм створює у свідомості команди фахівців цілісний, інтегральний образ системи, яку їм доведеться спільно розробляти. Наявність діаграм підвищує супроводжуваність програмного продукту. І справді, часто супровід виконують не ті самі фахівці, що розробляли ранні версії системи, а розібратися в чужому коді з нуля не так-то й просто. Наявність документації загалом пришвидшує вникання в зміст чужої роботи, а якщо ця документація графічна та недвозначна, то й поготів.

Всі діаграми мови UML поділяються на два типи – структурні та поведінкові.

Для того, щоб зобразити об'єктну структуру ПЗ, взаємозв'язок між її елементами,а також графічно представити предметну область засобами UML використовують діаграму класів.

2. Діаграма класів, її елементи. Діаграма класів – центральна ланка об’єктно-орієнтованих методів розробки програмного забезпечення.

Основним поняттям даної діаграми є класи та відношення між ними. Клас при цьому традиційно розуміють як сукупність загальних ознак заданої групи об'єктів предметної області. Відповідно до цього визначення на діаграмі класів кожному класу відповідає група об'єктів, загальні ознаки яких фіксує клас.

2.1. Клас в UML. Позначення класу в UML – прямокутник, розділений на 3 секції: назва класу, атрибути, операції.

Крім цього, часто під час абстрактного моделювання системи використовують для позначення класу звичайний прямокутник (без розбиття на секції), у якому вказують лише назву класу. Такий підхід використовується, якщо потрібно зобразити систему на вищому рівні абстракції, де не важливі деталі реалізації кожної сутності.

Рис 1. Схематичне предсталення поняття «Клас» в мові UML.

Розглянемо детальніше кожну секцію класу.

<u>Перша секція</u> містить назву класу. Приклад – CMammal, CHuman, CCat, CStudent, CBook. Застосування префіксу C – данина угорській нотації (C – перша літера слова class).

<u>Друга секція</u> містить перелік полів класу (атрибутів, даних), тобто, фрагментів істотних, важливих даних, які відрізняють один екземпляр класу від іншого. Прикладами даних класу CStudent є номер залікової книжки, номер студентського квитка, рік вступу тощо. Звичайно, ще студент має ім’я, прізвище, громадянство, дату народження, ідентифікаційний код тощо, але цими ж властивостями володіють і інші люди, незалежно від того, навчаються вони у

вищому навчальному закладі чи ні. Для класу CBook це буде назва книги, рік видання, кількість сторінок, ціна, перелік авторів, жанр тощо.

Клас може мати довільну кількість атрибутів або не мати їх взагалі. Наприклад, у будь-якої стіни є висота, ширина і товщина; при моделюванні клієнтів можна задавати прізвище, адресу, номер телефону й дату народження. Таким чином, атрибут є абстракцією даних об’єкта або його стану. У кожний момент часу атрибут об’єкта, який належить даному класу, має визначене значення.

Найпростішим чином поле класу описується так:

<модифікатор доступу> <ім’я поля> : <тип>

Приклади:

+m_iPageAmount: short int // кількість сторінок у книзі

+m_czTitle: string // назва книги

+m_czRecordBook: string // номер залікової книжки

У наведених прикладах використовуються такі модифікатори доступу:

- "+" – позначає атрибут загальнодоступний (public). Атрибут з цією зоною видимості доступний або видний з будь-якого іншого класу пакету, в якому визначена діаграма.
- "#" – позначає атрибут захищений (protected). Атрибут з цією зоною видимості недоступний або не видний для всіх класів, за винятком підкласів даного класу.
- "-" – позначає атрибут закритий (private). Атрибут з цією зоною видимості недоступний або невидний для всіх класів без виключення.
- "~" - позначає атрибут пакетний (package). Атрибут з цією зоною видимості недоступний або не видний для всіх класів за межами пакету, в якому визначений клас-власник даного атрибуту.

Буває, що в поля є значення за замовчанням. Наприклад, кіт має лапи, за замовчанням 4. Цей факт позначається так:

#m_iPawAmount: unsigned char = 4

Тут модифікатор доступу вибраний лише для формального прикладу, без прихованого змісту і зовсім не тому, що котячі лапи потрібно захищати.

Можливо, що властивість набуває одне з дискретних, наперед відомих значень, тобто, може бути описана переліченнями. Наприклад, у світлофорах використовуються усього три кольори. У тижні є рівно сім днів. Прийнято додавати суфікс Kind в назву перелічуваного типу: TrafficLightColorKind, WeekDayKind.

Може бути, що властивість має кілька значень (кілька імен, кілька номерів телефонів, кілька адрес електронної пошти). Для відображення цього факту застосовується кратність, що вказується після типу у квадратних дужках. Кратність може задаватися діапазоном, дискретними значеннями, переліченими через кому і/або з використанням символу «*», що позначає довільну кількість.

Наприклад:

2 – строго 2,

1..7 – від 1 до 7,

1, 12 – або 1, або 12

* - довільна кількість

1..* – від одиниці до безмежності

Приклад:

+m_czAuthorName: string[0..8]

Одна книга може мати декількох авторів (одного, двох, трьох,…). Книга може не мати автора. Більше певної здорової кількості пишуть «за редакцією такого-то», тобто, зводяться до одного автора.

Третя секція містить перелік операцій (методів) класу. Операція - це абстракція того, що дозволено робити з об’єктом. У всіх об’єктів класу є спільний набір операцій. Клас може містити довільне число операцій або не містити їх взагалі. Часто звернення до операції об’єкта змінює його стан або його дані. Операції класу зображаються в розділі, розміщеному нижче розділу з атрибутами. При цьому можна обмежитися тільки іменами.

Методи описуються так:

<модифікатор доступу> <ім'я методу> ([<параметр 1>, <параметр 2,…>]) : <тип результату>

Кожен з параметрів описується у той же спосіб, щоб й атрибути класу. Наприклад, той же кіт має метод Purr (муркотіти), який в якості параметру приймає позитивні емоції, а повертає звук:

-Purr(joy: Emotions): Sound

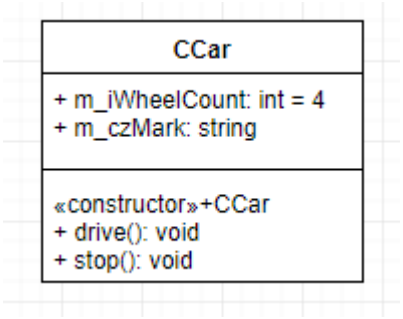
Метод теж може повертати значення за замовчанням. Пишеться воно теж після типу результату через знак рівності.

Для позначення конструктора використовується поняття стереотипу: «constructor»+Cat

Статичні атрибути/методи підкреслюються.

Допускається і спрощений опис методів: Purr().

Навіть якщо секції атрибутів і операцій порожні, в позначенні класу вони можуть бути виділені горизонтальною лінією, для того, аби відрізнити клас від інших елементів мови UML. Приклади графічного зображення конкретних класів наведені на рис. 2.



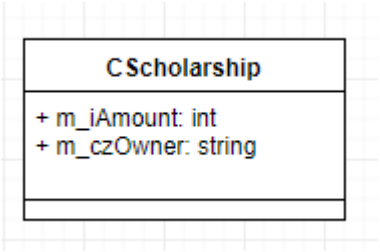


Рис 2. Приклади конкретних класів в мові UML

2.2. Поняття «Відношення» в діаграмі класів.

При побудові абстракцій класи рідко існують автономно. Як правило, вони різними способами взаємодіють між собою.

Існує чотири види відношень в об'єктно-орієнтованому моделюванні:

- *залежності*, які описують існуючі між елементами системи відношення використання;
- *узагальнення*, які зв'язують узагальнені класи з спеціалізованими;

- *асоціації*, які представляють структурні відношення між об’єктами (крім цього, виділяють ще два підвиди відношення «асоціації» - агрегація та композиція);
- *реалізація,* як середнє між *узагальненням* та *залежністю*.

Кожне з цих відношень дозволяє по-різному комбінувати сутності.

У табл. 1 наведено графічні позначення кожного з типів відношень.

Таблиця 1

Позначення	Назва
	Узагальнення
	Залежність
	Асоціація
	Агрегація
	Композиція
	Реалізація

Розглянемо кожне з відношень більш детально.

Відношення узагальнення на діаграмі класів описує ієрархічну будову класів і успадкування їх властивостей та поведінки. При цьому похідний (спеціалізований) клас має всі властивості та поведінку базового (узагальненого) класу, а також свої власні властивості та поведінку, які відсутні у базовому класі. Це відношення зображається суцільною лінією з трикутною стрілкою в сторону базового елемента (від потомка до батька).

Узагальнення (наслідування) є одним з трьох китів ООП. Мета його застосування – уникнення дублювання коду. Наприклад, з точки зору університету, студента характеризують ім’я, прізвище, по батькові, дата народження, адреса проживання, адреса електронної пошти та телефон, ідентифікаційний код, номер студентського квитка, номер залікової книжки, інститут/факультет і випускова кафедра, рік вступу, освітньо-кваліфікаційний

рівень, спеціальність, успішність тощо. У викладачеві університет цікавлять ім'я, прізвище, по батькові, дата народження, адреса проживання, адреса електронної пошти та телефон, ідентифікаційний код, наявність наукового ступеня, наявність вченого звання, науково-педагогічний стаж, посада, інститут/факультет і кафедра, де працює викладач тощо. Частина властивостей/даних є спільними. Тому варто усе спільне винести в окрему сутність – тут доречною буде назва «Людина», оскільки всі названі спільні властивості притаманні кожній людині. «Студент» і «Викладач» є частинним випадком «Людини», тобто, наслідують усі властивості та поведінку людини, але кожен додатково ще має своїми специфічними риси та поведінку. Інші приклади узагальнення показано на рис. 3.

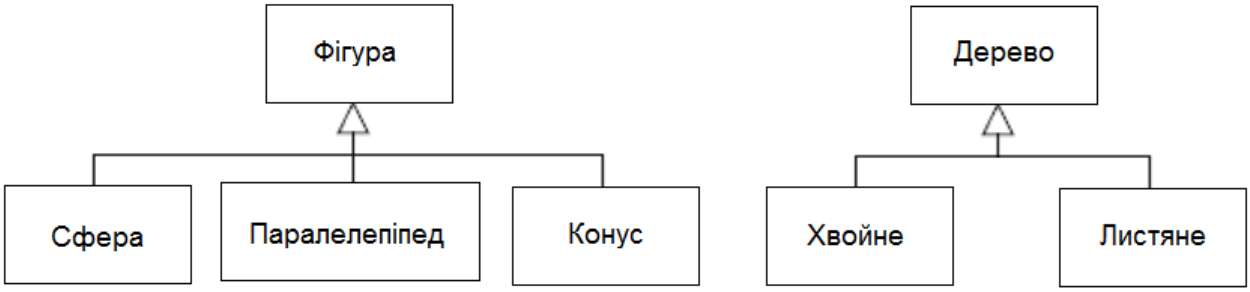


Рис. 3. Приклади узагальнення

Відношення залежності використовується в такій ситуації, коли деяка зміна одного елемента моделі вимагає зміни іншого, залежного від нього елемента моделі. Це відношення зображається штрих-пунктирною лінією із звичайною стрілкою в сторону незалежного елемента (від елемента-клієнта до елемента-джерела).

Наприклад, сутність «Продажі» залежіть від продукту, адже чим більша ціна продукту тим, зазвичай, меншу його кількість купують і навпаки (рис.4).

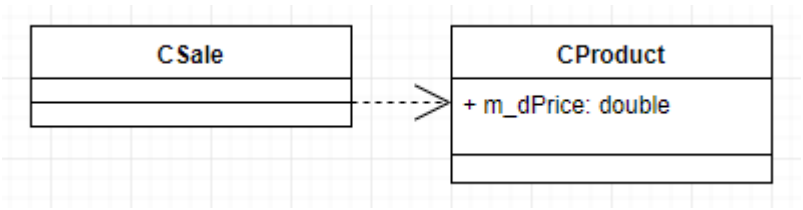


Рис.4. Приклад відношення «залежність»

Відношення залежності у діаграмі класів означає, що одна сутність використовує іншу якимось чином. Наприклад, розклад занять залежить від дисципліни. Зміниться обсяг дисципліни – доведеться міняти і розклад (додавати чи вилучати пари). Це може проявлятися в тому, що один з методів класу А приймає в якості параметру екземпляр класу В або викликає якийсь з методів класу В. Наприклад, якщо у класі А використано якийсь готовий метод сортування і згодом сигнатура цього методу змінилася, доведеться вносити зміни у клас А.

Увага! Часто студенти, коли їх просять показати залежність між якимись двома класами, називають площу кола та його радіус. Мовляв, площа кола залежить від радіусу. Це так, однак, радіус – не клас. Його можна виразити скалярним типом (дійсне число), для нього не потрібно заводити окремий тип даних. Площа – це теж скаляр. Немає змісту «обгортати у клас» одне-єдине поле. А от саме коло може бути представлене класом. Вочевидь, обов'язковими полями будуть координати центру та радіус. Тобто, радіус – властивість кола і атрибут відповідного класу. Площа могла б повертатися методом класу, який її обчислює на основі радіусу. Ім'я, вік, номер залікової книжки, ідентифікаційний код, номер банківського рахунку, баланс рахунку – це все приклади властивостей, а не окремих сутностей.

Відношення асоціації відповідає наявності деякого зв'язку між елементами моделі. Асоціація поєднує рівноправні сутності. Таке відношення зображається суцільною лінією і при потребі доповнюється спеціальними символами. Спеціальні символи характеризують окремі властивості конкретної абстракції.

В якості цих символів можуть бути 1) ім'я асоціації, 2)заповнена всередині стрілка-трикутник, яка вказує на порядок зв'язку, 3) імена елементів-ролей асоціації, 4) кратність цих ролей.

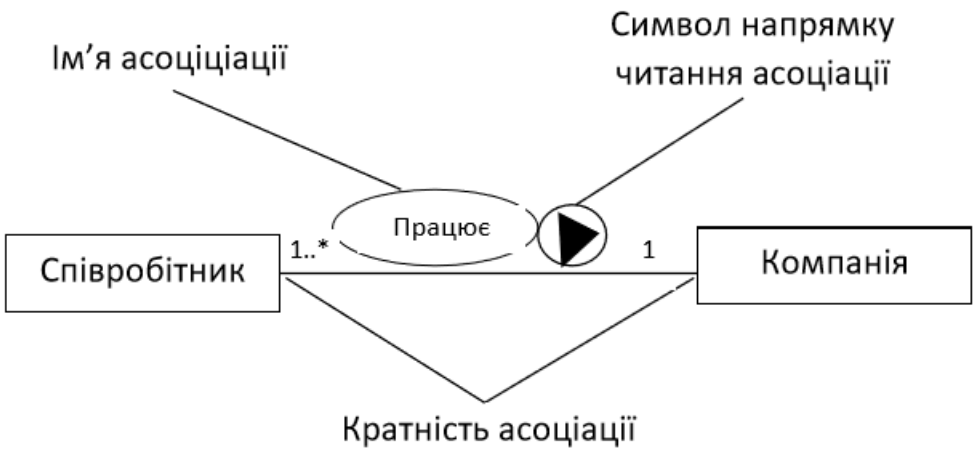


Рис. 5 Графічне зображення асоціації між класами

Приклади асоціації – «викладач- студент», «лікар – пацієнт», «літак – пілот», «літак –пасажир», «бібліотека – читач», «комп’ютер-користувач», …Кожна з названих пар пов’язані між собою, але жоден у парі не є ні нащадком, ні складовою іншого (рис. 6).

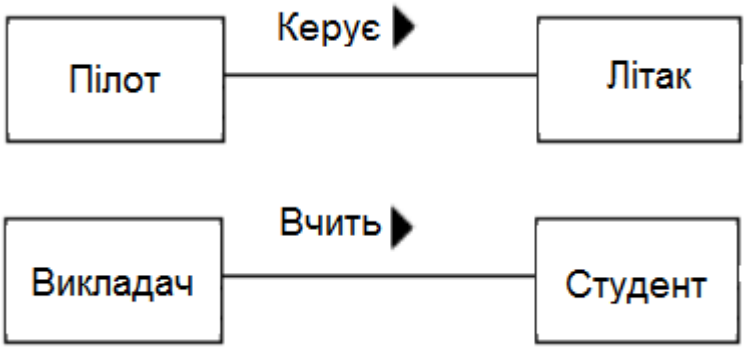


Рис. 6. Приклади асоціації

Крім цього, у відношенні асоціації може вказуватися кратність. Наприклад, студент вивчає різні дисципліни, а дисципліна вивчається різними студентами. Тому між студентом і дисципліною зв'язок «багато до багатьох». Студент має лише один студентський квиток. Якщо він загубиться, доведеться пройти виснажливу процедуру поновлення, в результаті якої старий квиток вважатиметься недійсним, а новий набуде чинності. Студентський квиток має лише одного власника. Це зв'язок «один до одного». Натомість, студент може мати максимум два закордонних паспорти, чинних одночасно. Кожен паспорт має

лише одного власника. Це зв'язок «один до двох». На рис. 7 показано кратності для розглянутих прикладів.

Рис. 7. Кратність у відношенні асоціації

Частковим випадком відношення асоціації є відношення *агрегації*, яке в свою чергу має спеціальну форму – відношення *композиції*.

Відношення агрегації має місце між деякими елементами в тому випадку, якщо один елемент є сутністю, яка включає в собі інші сутності в якості складових частин. Дане відношення має фундаментальне значення для опису структури складних систем, оскільки застосовується для представлення системних взаємозв’язків типу «частина-ціле». Таким чином утворюється деяка ієрархія, яка принципово відрізняється від тої, що породжується відношенням узагальнення. Частини системи ніяк незобов’язані наслідувати властивості і поведінку системи, оскільки є самостійними сутностями. Більш того, частини цілого мають свої атрибути і операції, які істотно відрізняються від атрибутів і операцій всього цілого. Графічно відношення агрегації зображається суцільною лінією, один з кінців якої є незаповненим всередині ромбом. Цей ромб вказує на той елемент, який «цілим» (системою). Приклади подані на рис. 8.

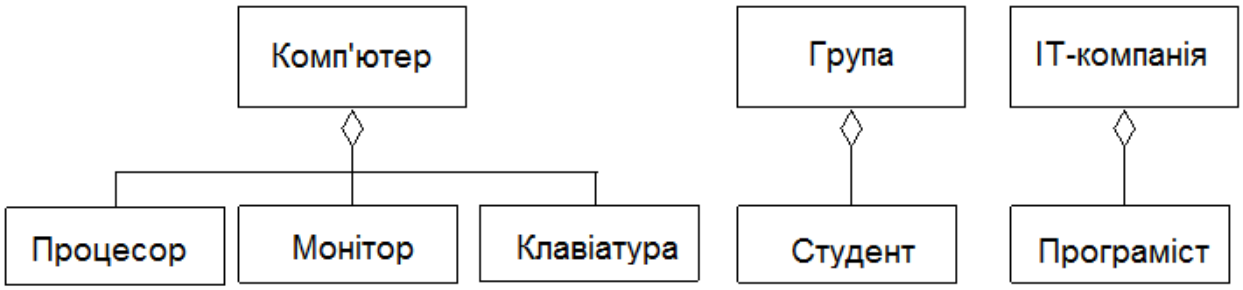


Рис. 8. Приклади агрегації

Відношення композиції (строга агрегація) є частковим випадком відношення агрегації. Специфіка цього взаємозв'язку заключається в тому, що частини не можуть виступати у відриві від цілого. Тобто із знищенням цілого знищуються і всі його складові частини. Відношення композиції зображається суцільною лінією, один з кінців якої є заповненим всередині ромбом. Цей ромб вказує на той елемент, який є «цілим» (системою), рис. 9.

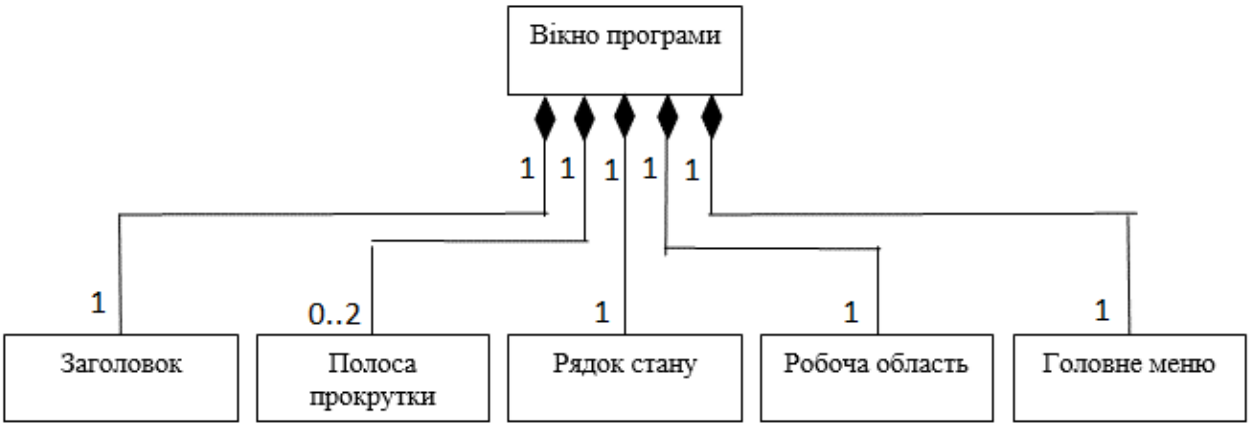


Рис. 9. Приклади композиції

У агрегації та композиції теж використовується кратність (рис. 9). Сутність «Вікно програми» може мати 1 заголовок, 2 полоси прокрутки (горизонтальну та вертикальну), 1 рядок стану, 1 робочу область і 1 головне меню.

З точки зору реалізації поданий вище приклад зі студентом і студентським квитком – приклад композиції. Фізично паспорт не є складовою людини, але у клас «Людина» зручно включити атрибутом екземпляр класу «Паспорт», де зібрано номер, кінцеву дату і т.д. Це за умови, що клас «Паспорт» використовується ще десь. Інакше можна просто «лінійно» розмістити названі поля у класі «Людина».

Якщо типом атрибуту класу А є клас В, має місце агрегація (або композиція), рис. 10.

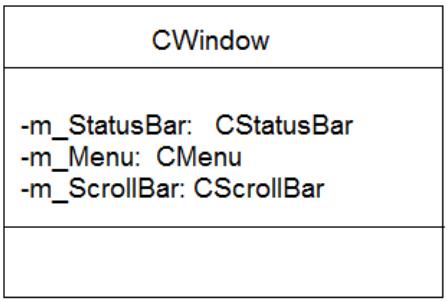


Рис. 10. Приклад композиції («Меню» є частиною сутності «Вікно»)

Увага! Типовою помилкою студента-першокурсника є зміна пріоритетів у визначенні композиції (агрегації). Частина не може існувати без цілого, це правда. Однак, першочерговим є відношення «частина – ціле». Якщо між двома сутностями відсутнє відношення «частина – ціле», то це не композиція (агрегація)! Навіть якщо одна сутність сильно залежить від іншої. Дуб не живе без ґрунту, але ґрунт не складається з дубів, а дуби з ґрунту.

Відношення реалізації пов'язане з поняттям інтерфейсу.

Часто ми не знаємо, як всередині влаштовані речі, якими користуємося щодня. Зокрема, практично кожен має мобільний телефон, але далеко не кожен знає, яким чином забезпечується передача голосу, за яким протоколом передаються текстові повідомлення чи навіть які частоти використовує цей пристрій. Однак, брак таких специфічних знань не заважає нам продовжувати успішно користуватися технікою. Візьмемо, до прикладу, пульт від телевізора чи банківська карта. Вони є свого роду інтерфейсами. Через пульт ми спілкуємося з телевізором (натискаємо потрібну кнопку – і міняється канал, а як це робиться – не наш клопіт). Банківська карта є «інтерфейсом до рахунку».

Інтерфейс по відношенню до класів – це сукупність усіх операцій, що надаються класом. Тобто, інтерфейс представляє видиму ззовні поведінку класу повністю або частково. З програмної точки зору інтерфейс – це сукупність сигнатур операцій (методів) класу (тільки сигнатур, ніякої справжньої реалізації!).

Графічно інтерфейс представляється у вигляді кола (кружечка, «льодяника»). Сам по собі він змісту не має, тому коло прикріплюється до класу, який представляє цей інтерфейс, рис 11.

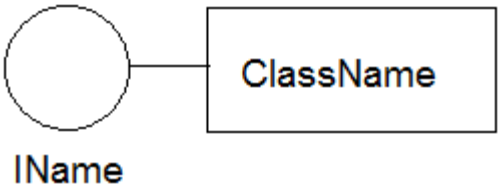


Рис. 11. Приклад реалізації

Прийнято імена інтерфейсів починати з літери «I».

Можливий, однак, і інший спосіб подання, рис. 12

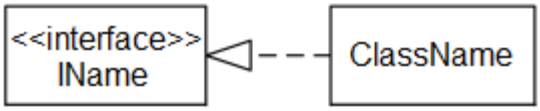


Рис. 12. Приклад реалізації

Стрілка у відношенні реалізації вказує на інтерфейс. Саме позначення відношення є своєрідним «гібридом» позначення залежності (лінія) та узагальнення (стрілка).

Отже, інтерфейс можна зобразити як клас зі стереотипом «interface».

2.3. Рівні діаграм класів

UML пропонує використати три рівні діаграм класів залежно від ступеня їхньої деталізації:

• *концептуальний рівень*, на якому діаграми класів, названі в цьому випадку контекстними, демонструють зв'язки між основними поняттями предметної області;

• *рівень специфікацій*, на якому діаграми класів відображають інтерфейси класів предметної області, тобто зв'язки об'єктів цих класів;

• *рівень реалізації*, на якому діаграми класів безпосередньо показують поля й операції конкретних класів.

Практично це три різних моделі, зв'язок між якими неоднозначний. Так, якщо концептуальна модель визначає деяке поняття предметної області як клас, то це

не означає, що для реалізації цього поняття буде використаний окремий клас. Хоча у всіх трьох моделях нас цікавлять типи об'єктів (класи) і їхні статичні відношення, що дозволяє використовувати єдину нотацію.

Кожну з перерахованих моделей використовують на конкретному етапі розробки програмного забезпечення:

• концептуальну модель – на етапі аналізу;

• діаграми класів рівня специфікації – на етапі проектування;

• діаграми класів рівня реалізації – на етапі реалізації.

Концептуальні моделі відповідно до визначення оперують поняттями предметної області, атрибутами цих понять і відношеннями між ними. Поняттю в предметній області розроблювального програмного забезпечення можуть відповідати як матеріальні предмети, так й абстракції, які застосовують фахівці предметної області.

Наприклад, на рис.13 зображено діаграму класів на концептуальному рівні, немає чітких визначених власивостей та методів кожної сутності, адже на діаграмі наведено лише основні поняття даної предметної області та взаємозв’язки між ними.

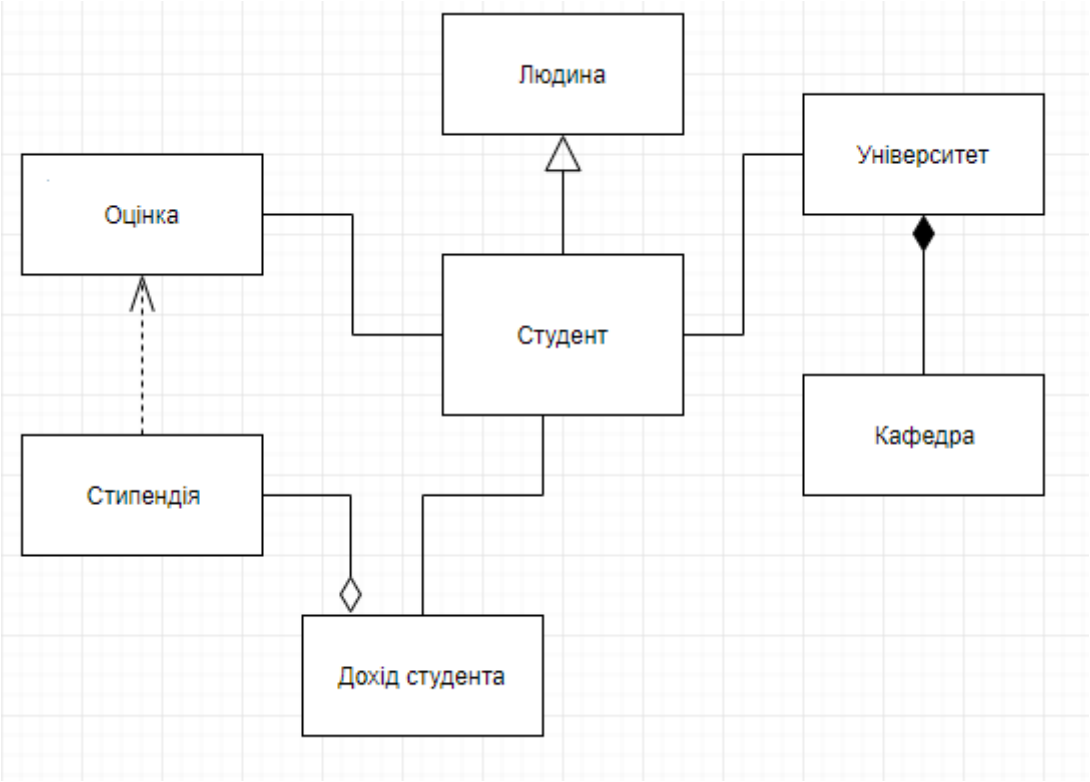


Рис 13. Приклад UML-діаграми на концептуальному рівні

На рівні реалізації класи описуються детально, вказуються атрибути, методи роботи з ними. Зазвичай діаграма класів на рівні реалізації розробляється не для всієї системи, а для певного функціоналу (через складність), рис. 14.

Для правильного розбиття на класи, виділення необхідних атрибутів та методів на рівні реалізації, необхідно дотримуватись також основних принципів ООП під час опису діаграм класів на *рівні реалізації*:

- інкапсуляція – приховування реалізації. Принцип ООП, який дає можливість створювати для користувача зручні об’єкти предметної області і оперувати ними, не вдаючись у деталі їхнього створення;
- поліморфізм – одна специфікація - багато реалізацій. Принцип ООП, який дає змогу створювати функції (методи) з одним і тим же ім'ям, але з різним програмним кодом (реалізація) в залежності від того, об'єкт якого класу використовується під час виклику даного методу;
- успадкування – побудова нового класу (дочірнього) на основі базового (батьківського). Варто зауважити, що дочірній клас отримує усі властивості батьківського класу, але окрім того містить додаткову функціональність. Використання даного принципу забезпечує оптимізацію коду, адже загальні властивості сутностей описуються в батьківському класі і не потребують повторного дублювання коду в дочірніх класах.

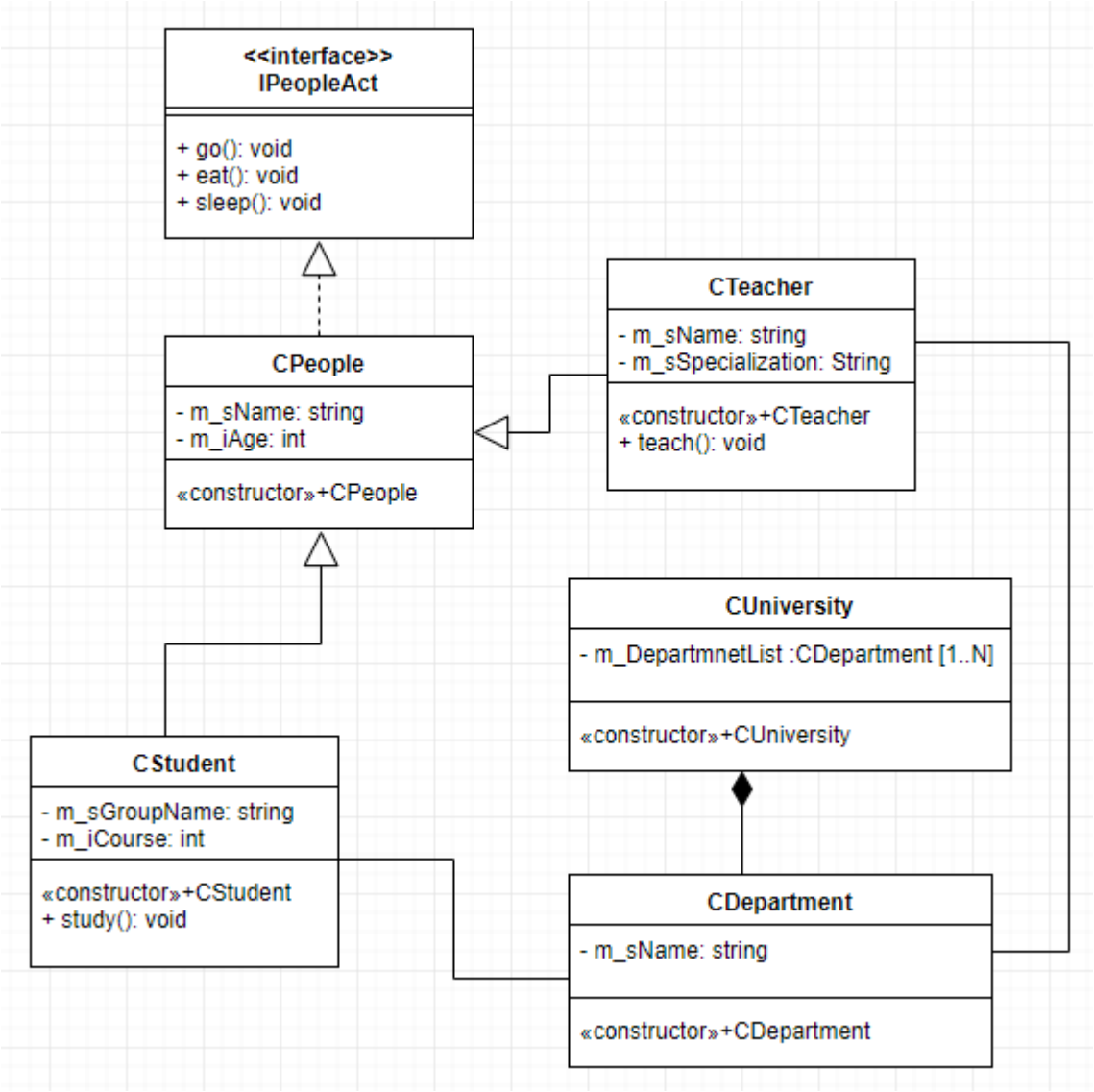


Рис 14. Приклад UML-діаграми на рівні реалізації

Приклад 1.

Розглянемо принципи ООП та деякі відношення між класами на прикладі сутності «легковий автомобіль».

- успадкування (відношення узагальнення)::

базова сутність «легковий автомобіль» (вміє їздити). Дочірні сутності «седан», «універсал», «хеч-бек» містять основні властивості (мотор, кузов, колеса) та поведінку (їздити, стояти) батьківського класу, але також кожен з дочірніх класів має свою специфіку;

- інкапсуляція:

водій під час їзди крутить кермо і натискає на педаль газу, не замислюючись, що в цей час відбувається з інжектором, дросельним клапаном і т.д. Тобто для водія частина реалізація автомобіля є «прихованою» для його ж зручності;

- поліморфізм:

будь-яке навчання водінню не мало б сенсу, якби людина, що навчилася їздити, скажімо, на ВАЗ 2106 не могла потім їздити на ВАЗ 2110 або BMW X5. З іншого боку, важко уявити людину, яка змогла б нормально керувати автомобілем, в якому педаль газу знаходиться лівіше педалі гальма, а замість керма - джойстик. Уся річ в тому, що основні елементи управління автомобіля мають одну і ту ж конструкцію і принцип дії. Водій точно знає, що для того, щоб повернути ліворуч, він повинен повернути кермо, незалежно від того, є там гідропідсилювач чи ні. Отже, можна сказати, що всі автомобілі мають один і той же інтерфейс, з яким вміє працювати водій, хоча реалізація інтерфейсу в кожному типі автомобіля різна;

- відношення композиція:

сутність «автомобіль», не може бути без мотору, кузову. Тобто це невід'ємні його частини;

- відношення агрегація:

автомобіль може бути з клімат-контролем, а може бути і без, адже в обох випадках авто виконує основне своє призначення. Тобто клімат-контроль є частиною авто, але не його основною;

- відношення асоціація:

сутність «автомобіль» асоціюється із сутністю «водій». Кратність такого відношення буде «багато до багатьох», адже один водій може мати декілька автомобілів і на одному авто можуть їздити декілька водіїв.

Приклад 2.

Розглянемо принципи ООП та деякі відношення між класами на прикладі сутності «Ручка».

- успадкування (відношення узагальнення)::

базова сутність «ручка» (вміє писати), від якої успадковують основні властивості та поведінку дочірні сутності: «кулькова ручка», «чорнильна ручка» (відрізняються принципами роботи). Всі нащадки ручки вміють писати, але мають свою специфіку;

- інкапсуляція:

кожна з ручок інкапсулює (приховує) свою реалізацію, тобто можна писати ручкою, але не знати як відбувається подача чорнила, гелю і т.д. з ручки на папір;

- поліморфізм

у базовій ручці описано її основну поведінку – «вміє писати», але те як саме, яким кольором і т.д. описано в конкретній реалізації базової сутності, тобто ми маємо одну специфікацію роботи ручки і багато реалізації як саме і чим це виконується;

- відношення композиція:

сутність «кулькова ручка», не може бути без стержня, так само як чорнильна ручка не може бути без резервуару для чорнила. Тобто це невід'ємні частини ручки;

- відношення агрегація:

ручка може бути з ковпачком, а може бути і без, вона в обох випадках виконує своє призначення. Тобто ковпачок є частиною ручки, але не основною;

- відношення асоціація:

сутність «ручка» асоціюється із сутністю «писар». Крім цього, варто зауважити, що кратність відношення буде «багато до багатьох», адже один писар може мати декілька ручок і однією ручкою можуть писати декілька писарів.

Висновки

Отже, мова UML є досить строгим і потужним засобом моделювання, яка може бути ефективно використана для побудови концептуальних, логічних і графічних моделей складних систем різного цільового призначення.

Однією з основних діаграм мови UML є діаграма класів – логічна модель, що відображає архітектурні аспекти структурної побудови складної системи.

Для правильної побудови діаграми класів необхідно чітко проаналізувати систему, яку моделюють, та враховувати принципи ООП.

Теми для повторення з дисципліни «ООП»

1. Принципи ООП: успадкування, поліморфізм, інкапсуляція.
2. Клас. Абстрактний клас. Параметризований клас. Об'єкт.
3. Операції над об'єктами: модифікатор, селектор, ітератор, конструктор, деструктор.

Завдання

Згідно індивідуального варіанту провести аналіз предметної області. Для виконання завдання:

1. Скласти словник предметної області.
2. Побудувати UML-діаграму класів на концептуальному рівні засобами програми Visio. Зобразити коментарі на схемі. Вказати відношення між сутностями (**узагальнення, звичайна асоціація, агрегація, композиція, залежність**) із обов'язковим зазначенням їх характеристик (кратність, назва асоціації і т.п.).
3. Побудувати UML-діаграму конкретних класів на рівні реалізації засобами програми Visio. Чітко вказати усі поля та методи класів з відповідними модифікаторами доступу, а також усі необхідні відношення між класами.
4. Оформити звіт.

Варіанти

1. Інформаційна система «Бібліотека».
2. Інформаційна система «Деканат».
3. Інформаційна система «Магазин електротехніки».
4. Інформаційна система «Кінотеатр».
5. Інформаційна система «Фітнес-центр».
6. Інформаційна система «Ресторан».

7. Інформаційна система «Школа».

8. Інформаційна система «Аеропорт».

9. Інформаційна система «Комп'ютерний сервіс».

10. Інформаційна система «Банк».

11. Інформаційна система «Готель».

12. Інформаційна система «Зоопарк».

Контрольні запитання

1. Що таке модель, моделювання? Розкрийте поняття «модель системи»?
2. Які завдання вирішує процес моделювання?
3. Дайте визначення поняттям «клас», «абстрактний клас», «об'єкт», «предметна область»
4. Охарактеризуйте принципи ООП.
5. Що таке модифікатор, селектор, ітератор, конструктор, деструктор?
6. Що таке словник даних предметної області? Як він будується?
7. Яке основне призначення UML?
8. Яка історія виникнення мови UML?
9. Опишіть типи сутностей мови UML.
10. Які діаграми мови UML ви знаєте?
11. Що таке «сутність», «відношення»?
12. Яке призначення діаграми класів?
13. Наведіть класифікацію діаграм класів за рівнем їхньої деталізації.
14. Що таке «атрибут класу», «операція»?
15. Які є відношення між класами, як вони зображаються?
16. Наведіть приклади усіх відношень мовою UML.
17. Наведіть приклади для усіх типів сутностей UML.
18. Як діаграма класів пов'язана з іншими діаграмами мови UML?